

15-213, 20xx年秋季数据实验室：位操作作业

布置日期：8月30日，截止日期：9月12日星期三，23:59

Harry Bovik (bovik@cs.cmu.edu) 是本次作业的主要负责人。

1. 简介

本次作业的目的是熟悉整型和浮点数的位级表示。你将通过解决一系列编程"谜题"来达到这个目的。这些谜题中有许多相当人为，但你在解决它们的过程中会发现自己更多地思考位操作。

2. 流程安排

这是一个个人项目。所有提交物均为电子版。说明和更正将发布在课程网页上。

3. 手册说明

站点特定：在此处插入一段说明讲师将如何向学生分发 datalab-handout.tar 文件的文本。

首先，将 datalab-handout.tar 复制到您计划进行工作的 Linux 机器上的（受保护的）目录中。然后输入命令：

```
unix> tar xvf datalab-handout.tar
```

这将导致在目录中解包一些文件。你唯一需要修改并提交的文件是 bits.c。

bits.c 文件包含每个整数谜题的框架。你的作业是使用仅限直线代码（即没有循环或条件）完成每个函数框架，并使用有限的 C 算术和逻辑运算符。具体来说，你只允许使用以下八个运算符：

```
! ~ & ^ | + << >>
```

其中一些函数进一步限制了此列表。此外，你不允许使用任何超过 8 位的常量。请参阅 bits.c 中的注释，了解详细的规则以及对期望编码风格的讨论。

4. 谜题

本节描述了您将在 bits.c 中解决的谜题。

表 1 按从易到难的顺序列出了谜题。"评分" 字段给出了谜题的难度评分（分数），而 "最大操作符" 字段给出了实现每个函数时允许使用的最大操作符数量。请参阅 bits.c 中对函数期望行为的更多详细说明。您还可以参考 tests.c 中的测试函数。这些被用作参考函数来表示您函数的正确行为，尽管它们不满足您函数的编码规则。

函数名	描述	难度评分	最大操作符数
bitXor(x,y)	使用 & 和 ~ 实现 $x \oplus y$	1	14
tmin()	最小的二补整数	1	4
isTmax(x)	仅当 x 是最大的二补整数时为 True	1	10
allOddBits(x)	仅当 x 的所有奇数位都设置为 1 时为 True	2	12

函数名	描述	难度评分	最大操作符数
negate(x)	不使用 - 运算符返回 -x	2	5
isAsciiDigit(x)	如果 $0x30 \leq x \leq 0x39$ ，则为 True	3	15
conditional(x,y,z)	与 $x ? y : z$ 相同	3	16
isLessOrEqual(x, y)	如果 $x \leq y$ ，则为 True，否则为 False	3	24
logicalNeg(x)	不使用 ! 运算符计算 !x	4	12
howManyBits(x)	在二补码中表示 x 所需的最少位数	4	90
floatScale2(uf)	返回浮点参数 f 的 $2 \times f$ 的位级等效值	4	30
floatFloat2Int(uf)	返回 (int)f 的位级等价，其中 f 是浮点数参数	4	30
floatPower2(x)	返回 2.0^x 的位级等价，其中 x 是整数	4	30

表 1: Datalab 谜题。 对于浮点数谜题，值 f 是一个浮点数，它与无符号整数 uf 具有相同的位表示。

对于浮点数谜题，你需要实现一些常见的单精度浮点数操作。对于这些谜题，你可以使用标准的控制结构（条件语句、循环），并且可以使用 int 和无符号数据类型，包括任意的无符号和整数常量。你不可以使用任何联合、结构或数组。最重要的是，你不可以使用任何浮点数数据类型、操作或常量。相反，任何浮点数操作数将被作为无符号类型传递给函数，并且任何返回的浮点数值将是无符号类型。你的代码应该执行位操作，以实现指定的浮点数操作。

附带的程序 fshow 帮助您理解浮点数的结构。要编译 fshow，切换到讲义目录并输入：

```
unix> make
```

您可以使用 fshow 查看任意模式作为浮点数代表什么：

```
unix> ./fshow 2080374784
```

浮点值 $2.658455992e+36$ 位表示 $0x7c000000$ ，符号 = 0，指数 = f8，小数部分 = 000000 规格化。
 $1.0000000000 \times 2^{(121)}$

您还可以给 fshow 十六进制和浮点数值，它会解析它们的位结构。

5. 评估

您的分数将根据以下分布计算，最高为 67 分：

- 36 分为正确性得分。
- 26 分为性能得分。
- 5 分为风格得分。

正确性得分。 您必须解决的谜题已被评为 1 到 4 的难度等级，其加权总和为 36 分。我们将使用 btest 程序评估您的函数，该程序将在下一节中描述。如果您的谜题通过 btest 执行的所有测试，您将获得全部分数，否则不得分。

性能点。 本课程现阶段的主要关注点是确保你能得到正确答案。然而，我们希望培养你尽可能保持简洁明了的意识。此外，有些谜题可以通过暴力破解解决，但我们希望你能更聪明。因此，我们对每个函数都设定了一个

允许使用的操作符数量上限。这个限制非常宽松，仅用于捕获极其低效的解决方案。对于每个满足操作符限制的正确函数，你将获得两个性能点。

风格分。 最后，我们为对您解决方案的风格和注释的主观评价预留了 5 分。您的解决方案应尽可能简洁明了。您的注释应具有信息量，但不必过于详尽。

3

自动评分你的作业

我们在讲义目录中包含了一些自动评分工具——`btest`、`dlc` 和 `driver.pl`——以帮助您检查您的工作的正确性。

• **btest:** 此程序检查 `bits.c` 中函数的功能正确性。要构建和使用它，请输入以下两个命令：

```
unix> make
unix> ./btest
```

请注意，每次修改你的 `bits.c` 文件时，都必须重新构建 `btest`。

逐个处理函数会很有帮助，边进行边测试每个函数。你可以使用 `-f` 标志指示 `btest` 仅测试单个函数：

```
unix> ./btest -f bitXor
```

您可以使用选项标志 `-1`、`-2` 和 `-3` 来向其提供特定的函数参数：

```
unix> ./btest -f bitXor -1 4 -2 5
```

请查看文件 `README` 以获取有关运行 `btest` 程序的文档。

• **dlc:** 这是来自 MIT CILK 小组的 ANSI C 编译器的修改版本，您可以使用它来检查每个谜题是否符合编码规则。典型用法是：

```
unix> ./dlc bits.c
```

该程序在未检测到问题时会静默运行，例如检测到非法操作符、操作符过多，或在整数谜题中存在非直线代码。使用 `-e` 开关运行：

```
unix> ./dlc -e bits.c
```

会导致 `dlc` 打印每个函数使用的操作符数量。输入 `./dlc -help` 获取命令行选项列表。

• **driver.pl:** 这是一个驱动程序，它使用 `btest` 和 `dlc` 来计算您解决方案的正确性和性能点。它不接受任何参数：

```
unix> ./driver.pl
```

您的讲师将使用 driver.pl 来评估您的解决方案。

6. 提交说明

站点特定：在此处插入文本，告诉每位学生如何提交他们的 bits.c 解决方案文件。

7. 建议

- 不要在 bits.c 文件中包含 `<stdio.h>` 头文件，因为它会使 dlc 混乱，并导致一些不太直观的错误消息。你仍然可以在 bits.c 文件中使用 `printf` 进行调试，而无需包含 `<stdio.h>` 头文件，尽管 gcc 会打印一条警告信息，你可以忽略它。
- dlc 程序比 C 语言或 gcc 强制更严格的 C 声明形式。特别是，任何声明都必须出现在任何非声明语句之前的一个代码块（你用花括号包围的部分）中。例如，它会对以下代码发出警告：

```
int foo(int x) {  
    int a = x;  
    a *= 3;      /* 非声明语句 */  
    int b = a;   /* ERROR: 此处不允许声明 */  
}
```

8. "超越教授"竞赛

为了乐趣，我们提供一项可选的"击败教授"竞赛，让你与其他学生和讲师竞争，开发出最高效的谜题。目标是使用最少的操作符来解每个数据实验室谜题。每个谜题中操作符数量与讲师匹配或超越的学生都是赢家！

要提交你的参赛作品到竞赛，请输入：

```
unix> ./driver.pl -u 'Your Nickname'
```

昵称最多 35 个字符，可以包含字母数字、撇号、逗号、句号、连字符、下划线和与号。你可以随意提交多次。你最新的提交将出现在实时排行榜上，仅以你的昵称标识。你可以通过将浏览器指向以下网址查看排行榜：

```
http://$SERVER_NAME:$REQUEST_PORT
```

站点特定：将 `$SERVER_NAME` 和 `$REQUEST_PORT` 替换为你在 `./contest/Contest.pm` 文件中设置的值。